The `crossOrigin` attribute has two possible values: `anonymous` and `user-credentials`. If `crossOrigin` is set to `user-credentials`, any user credentials for that origin are included in the request. For example, if the origin has cookies, the cookies will be included with the image request. This is similar to setting the `withCredentials` property on the `XMLHttpRequest` object. Setting the `crossOrigin` attribute to `anonymous` will still make the request, but there won't be any cookies attached to the request. It's best to always use the `anonymous` value, unless you absolutely need the user's cookies to load the image.

Once the image's cross-origin property is set and the request is successful, the image can be manipulated in the same way as a same-origin image. That means that the `toBlob`, `toDataURL`, and `getImageData` methods will work on a canvas with a cross-origin image.

Note that the `crossOrigin` attribute alone isn't enough to avoid tainting the canvas. As with most of the other CORS features we've covered in this chapter, the server must indicate that cross-origin access to the image is allowed. If the server doesn't allow it, the image will still be displayed in the canvas, but none of the data extraction methods will work.

## 2.6 CORS requests from jQuery

JQuery is a popular JavaScript framework that powers many JavaScript apps. JQuery has a function named `ajax` for making HTTP requests. Under the hood, the `ajax` method uses the `XMLHttpRequest` object to make HTTP requests. The following listing modifies the Flickr API to use jQuery.

**Listing 2.10 Using jQuery to make a cross-origin request**

```
<!DOCTYPE html>
<html>
<body>
<div id="photos">Loading photos...</div>
<script src="http://ajax.googleapis.com/ajax/libs/jquery/2.1.1/
     jquery.min.js"></script>
<script>
function loadPhotos() {
  var api_key = '<YOUR API KEY HERE>';
  var method = 'GET';
  var url = 'https://api.flickr.com/services/rest/?' +
      'method=flickr.people.getPublicPhotos&' +
      'user_id=32951986%40N05&' +
      'extras=url_q&format=json&nojsoncallback=1&' +
      'api_key=' + api_key;

  $.ajax(url, {
    type: method,
    dataType: 'json',
    success: function(data, textStatus, jqXHR) {
      if (data.stat == 'ok') {
        $('#photos').empty();
```

**$.ajax method both initializes and sends an HTTP request.**

**json data type indicates that response should be parsed as JSON.**

**success property is used instead of onload.**